up the tests, makes the issue easy to see rather than burying it among dozens of fixtures, and makes the correctness of the tests themselves easier to verify.

Dates and Times

Date and time logic has a well-deserved reputation as some of the most complex and irritating logic in any application. Testing calendar logic—including time-based reports, automatic logouts, and "1 day ago" text displays—can be a particular headache, but you can do a couple of things to simplify the time-logic beast.

Part of the Problem

You have a test that uses factories. You'd like to test some time-based code that might be used in a search or report result. This goes in spec/unit/project_spec.rb:

```
let(:runway) { create(:project, name: "Project Runway",
  start_date: "2018-01-20" }
let(:greenlight) { create(:project, name: "Project Greenlight",
  start_date: "2018-02-24" }
let(:gutenberg) { create(:project, name: "Project Gutenberg",
  start_date: "2018-01-31" }

it "finds recently started projects" do
  actual = Project.find_recently_started(6.months)
  expect(actual.size).to eq(3)
end
```

Here's code that makes the test pass, from app/models/project.rb:

```
def self.find_recently_started(time_span)
  old_time = Date.today - time_span
  all(conditions: ["start_date > ?", old_time.to_s(:db)])
end
```

On January 20, 2018, the test passes. And on January 21 it will pass, and it will pass the day after that....

Six months later, about June 20 (when you've probably long forgotten about this test, this sample data, and maybe even this entire project), the test will fail. And you'll spend way too much time trying to figure out what happened, until you remember the date issue and realize that the project with a January 20 date is no longer within the six-month time span specified in the test. Of course, changing all the dates just pushes the problem forward and gives us time to forget all about it again.

This issue may sound silly, but like many of the more ridiculous examples in the book, this happened to me and can end up costing a lot of time. I've